

TECHNICAL SPECIFICATION

E-Commerce SaaS Platform

=====

This document describes the functional and non-functional requirements of the e-commerce platform as agreed during the discovery phase. Each section corresponds to a distinct deliverable module.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog)

--- Module: ZephyrAuth ---

Description: Authentication and authorisation subsystem.

The ZephyrAuth module is responsible for handling all aspects of authentication and authorisation subsystem. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All ZephyrAuth operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules

--- Module: NebulaPay ---

Description: Payment processing engine with stripe integration.

The NebulaPay module is responsible for handling all aspects of payment processing engine with Stripe integration. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All NebulaPay operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terrafo

--- Module: AuroraCache ---

Description: Distributed caching layer backed by redis.

The AuroraCache module is responsible for handling all aspects of distributed caching layer backed by Redis. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All AuroraCache operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform module

--- Module: StellarAPI ---

Description: Public restful api gateway with rate limiting.

The StellarAPI module is responsible for handling all aspects of public RESTful API gateway with rate limiting. It exposes a dedicated REST API surface and integrates with the core platform via

internal service-to-service calls. All StellarAPI operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform mo

--- Module: CosmosSearch ---

Description: Full-text search integration via elasticsearch.

The CosmosSearch module is responsible for handling all aspects of full-text search integration via Elasticsearch. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All CosmosSearch operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terr

--- Module: OrionMetrics ---

Description: Analytics and kpi reporting dashboard.

The OrionMetrics module is responsible for handling all aspects of analytics and KPI reporting dashboard. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All OrionMetrics operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting

--- Module: PulsarNotify ---

Description: Real-time notification engine (email + push).

The PulsarNotify module is responsible for handling all aspects of real-time notification engine (email + push). It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All PulsarNotify operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies.

All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terrafor

--- Module: VegaExport ---

Description: Data export module - csv, excel, and pdf reports.

The VegaExport module is responsible for handling all aspects of data export module - CSV, Excel, and PDF reports. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All VegaExport operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terra

--- Module: SiriusAdmin ---

Description: Administrative back-office and user management panel.

The SiriusAdmin module is responsible for handling all aspects of administrative back-office and user management panel. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All SiriusAdmin operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directly with backend microservices. Load testing is performed with Locust targeting 500 concurrent virtual users at

peak. The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code u

--- Module: LyraCI ---

Description: Ci/cd pipeline and deployment automation.

The LyraCI module is responsible for handling all aspects of CI/CD pipeline and deployment automation. It exposes a dedicated REST API surface and integrates with the core platform via internal service-to-service calls. All LyraCI operations are logged at INFO level for audit trails. Performance SLA: p99 latency < 200 ms at 100 rps.

The system must handle concurrent requests from multiple tenants without performance degradation. Each service component is independently scalable via Kubernetes horizontal pod autoscaling. Database migrations are managed through Alembic with zero-downtime strategies. All API endpoints are versioned under /api/v1 and protected by JWT bearer tokens. Observability is implemented with structured JSON logs (structlog), distributed tracing (OpenTelemetry), and metrics exported to Prometheus. The infrastructure is defined as code using Terraform modules targeting AWS ECS Fargate. Staging and production environments are fully isolated with separate VPCs and IAM roles. Automated security scans (Trivy, Bandit) run on every pull request. The frontend communicates exclusively through the API gateway, never directl